Gurobi: recept na praktický test

Pro Jirku • Od formulace ILP k programu • 2. listopadu 2026

Cíl: zvládnout stále stejný postup při programování úloh podobných horníkům, barvení domů a rozmísťování jezdců. Základní formulaci už umíš; tady se soustředíme na její překlad do kódu.

Pořadí, které dodržuj pokaždé

1. Načti vstup. Zapiš si význam každého čísla, seznamu a dvojice. Ujasni si indexy.

2. Založ model. Vždy stejný import a stejný název model.

3. Vytvoř proměnné. Jednou větou si napiš, co znamená x[i] = 1 nebo x[i, j] = 1.

4. Přidej omezení. Každou větu zadání přelož zvlášť. Před blok dej komentář.

5. Nastav cíl. Co přesně sčítáš a zda to chceš maximalizovat, nebo minimalizovat.

6. Spusť řešení. Zavolej model.optimize() a zkontroluj model.Status.

7. Zapiš výstup. Teprve teď čti .X. Dodrž přesně formát ze zadání.

Domluvený styl v celém taháku

import gurobipy as gp • **model** • proměnné pomocí **addVars** • omezení pomocí **for + addConstr** • součty pomocí **gp.quicksum** • vnitřní indexy od **0**.

Žádná jedna šablona neodvodí správný model pro libovolné zadání. Opakovat ale můžeš strukturu programu a většinu zdejších vzorů. Mění se význam proměnných, omezení a formát souborů.

Jak číst: nejdřív strany 2-5, potom celý program na stranách 6-7. Strany 8-11 jsou zásobník vzorů. Strany 12-13 otevři při ladění a opakování.

1 / Přesná syntaxe a proměnné

```
import gurobipy as gp
model = gp.Model()

# Jedna promenna: addVar (bez s).
y = model.addVar(vtype=gp.GRB.BINARY, name="y")

# Cela rada promennych: addVars (se s).
x = model.addVars(n, vtype=gp.GRB.BINARY, name="x")

# Tabulka n krat k: pristupuj pres a[i, j].
a = model.addVars(n, k, vtype=gp.GRB.BINARY, name="a")
```

addVar vrací jednu proměnnou. **addVars** vrací kolekci proměnných přístupnou pomocí indexů. Při **addVars(n, k)** vzniknou všechny dvojice indexů od 0 do n-1 a od 0 do k-1.

Tři typy, které potřebuješ

```
# Ano/ne: 0 nebo 1.
x = model.addVars(n, vtype=gp.GRB.BINARY, name="x")

# Pocty kusu: cela nezaporna cisla.
q = model.addVars(n, vtype=gp.GRB.INTEGER,
                  lb=0, name="q")

# Spojite hodnoty: napr. cas zacatku.
t = model.addVars(n, vtype=gp.GRB.CONTINUOUS,
                  lb=0, name="t")
```

lb = dolní mez, **ub** = horní mez. Například **ub=10** dovolí nejvýše 10. Výchozí dolní mez je 0; pro proměnnou, která smí být záporná, musíš meze změnit. Typ piš vždy výslovně.

Jedno omezení = addConstr, vždy bez s

```
model.addConstr(x[0] + x[1] <= 1)

for i in range(n):
    model.addConstr(q[i] <= 10)
```

Cyklus zde zavolá **addConstr** n-krát. Metoda **addConstrs** také existuje, ale v tomto receptu ji nepotřebuješ. Není chyba používat jednotné číslo uvnitř cyklu.

Pozor na velikost písmen: addConstr, addVars, setObjective, optimize, GRB.BINARY. **name=** je jen jméno pro čitelnost a ladění, nemění matematiku.

2 / Součty a cykly bez zmatku

Jeden součet se překládá jedním quicksum

```
# Soucet vsech x[i].
gp.quicksum(x[i] for i in range(n))

# Soucet vah vybranych prvku.
gp.quicksum(w[i] * x[i] for i in range(n))

# Soucet pres celou tabulku.
gp.quicksum(x[i, j]
            for i in range(n) for j in range(k))
```

Výraz před **for** říká, co se sčítá. Část za **for** říká, přes které indexy. Samotný quicksum jen vytvoří výraz; omezení z něj udělá až **addConstr** s porovnáním.

„Pro každého horníka nejvýše jedno místo“

```
for m in range(M):
    model.addConstr(
        gp.quicksum(x[m, l] for l in range(L)) <= 1
    )
```

Vnější for: vybere horníka m a vytvoří jeho vlastní omezení. **Vnitřní for:** sečte všechna místa tohoto horníka. Pro M=3 a L=2 vznikne:

```
x[0, 0] + x[0, 1] <= 1
x[1, 0] + x[1, 1] <= 1
x[2, 0] + x[2, 1] <= 1
```

„Na každém místě nejvýše jeden horník“

```
for l in range(L):
    model.addConstr(
        gp.quicksum(x[m, l] for m in range(M)) <= 1
    )
```

Otázka před psaním: pro koho platí jedno omezení? Tento index dej do vnějšího cyklu. Co v něm sčítáš? Tento index dej dovnitř quicksum.

Nezaměň: součet přes všechny horníky i místa s mezí 1 by dovolil jen jedno přiřazení v celém dole.

3 / Zásobník běžných omezení

V této části jsou x[i], x[u] a x[v] binární. U přiřazování znamená a[i, j] = 1, že objekt i dostal možnost j. Jednotlivé ukázky jsou samostatné vzory.

Počet vybraných prvků

```
model.addConstr(gp.quicksum(x[i] for i in range(n)) <= K)
model.addConstr(gp.quicksum(x[i] for i in range(n)) >= K)
model.addConstr(gp.quicksum(x[i] for i in range(n)) == K)
```

První řádek = nejvýše K, druhý = alespoň K, třetí = právě K. **Vyber správný řádek, nepřidávej automaticky všechny tři.**

Rozpočet nebo kapacita

```
model.addConstr(
    gp.quicksum(w[i] * x[i] for i in range(n)) <= capacity
)
```

w[i] je známá spotřeba jednoho vybraného prvku. Při celočíselném počtu kusů nahradíš x[i] proměnnou q[i].

Dvojice a skupiny

```
# Dva prvky nesmeji byt vybrany soucasne.
for u, v in edges:
    model.addConstr(x[u] + x[v] <= 1)

# Je-li vybran u, musi byt vybran i v.
model.addConstr(x[u] <= x[v])

# Z dane skupiny musi byt vybran alespon jeden.
model.addConstr(gp.quicksum(x[i] for i in group) >= 1)
```

Přiřazení a zákaz podle vstupních dat

```
# Kazdy objekt dostane prave jednu moznost.
for i in range(n):
    model.addConstr(
        gp.quicksum(a[i, j] for j in range(k)) == 1
    )

# Zakaz nepovolene kombinace (allowed jsou vstupni data).
for i in range(n):
    for j in range(k):
        if not allowed[i][j]:
            model.addConstr(a[i, j] == 0)
```

4 / Vstup, spuštění a výsledek

Vstupní soubor čti podle řádků zadání

```
#!/usr/bin/env python3
import sys
import gurobipy as gp

with open(sys.argv[1], "r") as f:
    n, k = map(int, f.readline().split())
    weights = list(map(int, f.readline().split()))
```

readline() načte jeden řádek, **split()** ho rozdělí podle mezer, **int** převede text na číslo. **map** provede převod pro všechny části. Pro opakovaný přístup přes index použij **list**.

```
python3 main.py input.txt output.txt
```

sys.argv[0] = main.py, **[1]** = vstup, **[2]** = výstup. V notebooku tyto argumenty obvykle nemáš; pro test trénuj soubor main.py. Shebang dej opravdu na první řádek, pokud ho kontrola BRUTE vyžaduje.

Spuštění řešiče a kontrola stavu

```
model.optimize()

if model.Status == gp.GRB.OPTIMAL:
    print(model.ObjVal)
elif model.Status == gp.GRB.INFEASIBLE:
    print("Model nema pripustne reseni.")
else:
    raise RuntimeError(f"Neocekavany stav: {model.Status}")
```

OPTIMAL znamená nalezené optimum v tolerancích řešiče. **INFEASIBLE** znamená prokázanou nesplnitelnost. **TIME_LIMIT** neznamená nesplnitelnost. **SolCount > 0** znamená, že řešení existuje k přečtení, ale samo o sobě nepotvrzuje jeho optimalitu.

Výstup čti až po úspěšném řešení

```
# Pouzij jen ve vetvi s dostupnym resenim.
selected = []
for i in range(n):
    if x[i].X > 0.5:
        selected.append(i)

with open(sys.argv[2], "w") as f:
    f.write(" ".join(map(str, selected)) + "\n")
```

x[i] je objekt proměnné, **x[i].X** její nalezená hodnota. U binární proměnné používej **> 0.5**. Celočíselný výsledek převáděj **round(...)**; spojité výsledky bez požadavku zadání nezaokrouhluj.

5 / Celý program: barvení domů

Část 1 ze 2. Tento kód a kód na další straně tvoří jeden soubor main.py. Vychází přímo z tvého zadání Painting the neighborhood. Uvnitř počítáme od 0, vstupní čísla domů jsou od 1.

```
#!/usr/bin/env python3
import sys
import gurobipy as gp

# 1. VSTUP: n domu, m hran, k barev
with open(sys.argv[1], "r") as f:
    n, m, k = map(int, f.readline().split())
    cost = []
    volume = []
    for j in range(k):
        cj, vj = map(int, f.readline().split())
        cost.append(cj)
        volume.append(vj)

    edges = []
    for _ in range(m):
        u, v = map(int, f.readline().split())
        edges.append((u - 1, v - 1))

# 2. MODEL
model = gp.Model()

# 3. PROMENNE: x[i, j] = 1, kdyz dum i ma barvu j
x = model.addVars(n, k, vtype=gp.GRB.BINARY, name="x")

# 4a. Kazdy dum ma prave jednu barvu
for i in range(n):
    model.addConstr(
        gp.quicksum(x[i, j] for j in range(k)) == 1
    )

# 4b. Neprekrocime mnozstvi zadne barvy
for j in range(k):
    model.addConstr(
        gp.quicksum(x[i, j] for i in range(n)) <= volume[j]
    )

# 4c. Sousede nesmeji pouzit stejnou barvu
for u, v in edges:
    for j in range(k):
        model.addConstr(x[u, j] + x[v, j] <= 1)
```

Proč dvě smyčky u sousedů? Pro každou hranu zakazujeme společné použití každé konkrétní barvy. Různé barvy sousedů zůstávají povolené.

6 / Celý program: cíl a výstup

Část 2 ze 2. Připoj ji hned za předchozí kód. Barvy ve výstupu musí být od 1 do k, proto zapisujeme j + 1.

```
# 5. CIL: minimalni cena spotrebovane barvy
model.setObjective(
    gp.quicksum(cost[j] * x[i, j]
                for i in range(n) for j in range(k)),
    gp.GRB.MINIMIZE
)

# 6. RESENI
model.optimize()

# 7. VYSTUP: barva kazdeho domu, jedna na radek
if model.Status == gp.GRB.OPTIMAL:
    answer = []
    for i in range(n):
        for j in range(k):
            if x[i, j].X > 0.5:
                answer.append(j + 1)
                break

    with open(sys.argv[2], "w") as f:
        for color in answer:
            f.write(str(color) + "\n")

elif model.Status == gp.GRB.INFEASIBLE:
    with open(sys.argv[2], "w") as f:
        f.write("-1\n")
else:
    raise RuntimeError(f"Neocekavany stav: {model.Status}")
```

Co ověřit na příkladech ze zadání

Příklad 1: minimální cena je 19. **Příklad 2:** výstup je -1. **Příklad 3:** minimální cena je 42, čtyři domy mají první barvu a jeden druhou. Optimálních přiřazení může být více.

Do výstupního souboru nepiš cenu! Toto zadání požaduje pouze n řádků s barvami. model.ObjVal si můžeš vypsat do terminálu při ladění. U horníků je formát jiný: nejprve hodnota cíle, pak přiřazení.

Pro trénink: přepiš program ručně. Potom zakryj bloky 4a-4c a doplň je pouze podle českých komentářů.

7 / Jezdci: zakázané dvojice

U jezdců x[r, c] = 1 znamená, že na políčku stojí jezdec. Věže jsou pevně zadané. Podle tohoto zadání zakazují celý svůj řádek a sloupec. Následuje modelová část; **rooks** je seznam dvojic (řádek, sloupec) od 0.

```
model = gp.Model()
x = model.addVars(8, 8, vtype=gp.GRB.BINARY, name="x")

# Vez zakazuje cely radek i sloupec.
for rr, cc in rooks:
    for r in range(8):
        model.addConstr(x[r, cc] == 0)
    for c in range(8):
        model.addConstr(x[rr, c] == 0)

# Pouze 4 smery: kazdou neorientovanou dvojici jednou.
moves = [(1, 2), (1, -2), (2, 1), (2, -1)]
for r in range(8):
    for c in range(8):
        for dr, dc in moves:
            rr, cc = r + dr, c + dc
            if 0 <= rr < 8 and 0 <= cc < 8:
                model.addConstr(x[r, c] + x[rr, cc] <= 1)

model.setObjective(
    gp.quicksum(x[r, c]
                for r in range(8) for c in range(8)),
    gp.GRB.MAXIMIZE
)
model.optimize()
```

Převod souřadnic

```
# Vstupni text napr. "b2" -> radek 1, sloupec 1.
position = "b2"
c = ord(position[0]) - ord("a")
r = int(position[1]) - 1

# Opacny prevod vybraneho policka na vystup.
position = chr(ord("a") + c) + str(r + 1)
```

Ve výstupu napiš počet vybraných polí a potom každé políčko na samostatný řádek. Bez věží musí optimum vyjít 32. Pokud jsou všechna políčka zakázaná, optimum je 0, nikoli nesplnitelnost.

Přenositelný vzor: najdi konfliktní dvojice a pro každou přidej x[u] + x[v] <= 1. Stejný princip použiješ pro nezávislou množinu.

8 / Horníci: přiřazení a cechy

Vstupy podle fotografie: **M, L, F, U**; seznamy **g, s, d, h, p**; dvojice **friends**. Horníci i místa už mají indexy od 0. x[m, l] znamená přiřazení, y[m] znamená přítomnost horníka v dole.

```
model = gp.Model()
x = model.addVars(M, L, vtype=gp.GRB.BINARY, name="x")
y = model.addVars(M, vtype=gp.GRB.BINARY, name="y")

# Hornik ma nejvyse jedno misto; y rika, zda nejake ma.
for m in range(M):
    model.addConstr(
        gp.quicksum(x[m, l] for l in range(L)) == y[m]
    )

# Na miste je nejvyse jeden hornik.
for l in range(L):
    model.addConstr(
        gp.quicksum(x[m, l] for m in range(M)) <= 1
    )

# Velky hornik se do maleho mista nevejde.
for m in range(M):
    for l in range(L):
        if s[m] > d[l]:
            model.addConstr(x[m, l] == 0)

# Kazdy cech ma nejvyse tolik lidi jako zbyle dva spolu.
for guild in range(3):
    own = gp.quicksum(y[m] for m in range(M)
                      if g[m] == guild)
    other = gp.quicksum(y[m] for m in range(M)
                        if g[m] != guild)
    model.addConstr(own <= other)
```

Proč rovnost součtu s y[m]? Součet může být jen 0 nebo 1, protože y[m] je binární. Tím současně zajistíme nejvýše jedno místo a přesně propojíme přítomnost s přiřazením.

own a other nejsou nové proměnné. Jsou to Pythonové názvy pro lineární výrazy. Nová rozhodovací proměnná vzniká přes addVar nebo addVars.

if g[m] == guild filtruje horníky podle známého vstupu. Je to v pořádku. Stejně tak test velikosti s[m] > d[l]. O hodnotách rozhodovacích proměnných při stavbě modelu takto rozhodovat nemůžeš.

9 / Přátelství a podmínka NEBO

Pokračování modelu horníků. Pro každou dvojici přátel vytvoříme z[fidx] = 1 právě tehdy, když jsou přítomní oba. Nemusí stát vedle sebe; zadání chce jen jejich přítomnost v dole.

```
z = model.addVars(F, vtype=gp.GRB.BINARY, name="z")

for fidx, (a, b) in enumerate(friends):
    model.addConstr(z[fidx] <= y[a])
    model.addConstr(z[fidx] <= y[b])
    model.addConstr(z[fidx] >= y[a] + y[b] - 1)

production = gp.quicksum(p[m] * y[m] for m in range(M))
happiness = (
    gp.quicksum(h[m] * y[m] for m in range(M))
    + 5 * gp.quicksum(z[fidx] for fidx in range(F))
)

# Alespon jedna podminka: production >= U, happiness >= U.
choose = model.addVar(vtype=gp.GRB.BINARY, name="choose")
model.addConstr((choose == 0) >> (production >= U))
model.addConstr((choose == 1) >> (happiness >= U))

model.setObjective(production + happiness, gp.GRB.MAXIMIZE)
model.optimize()
```

Proč tři omezení pro společnou přítomnost?

Je-li některý přítel nepřítomný, první nebo druhé omezení vynutí z = 0. Jsou-li přítomní oba, třetí vynutí z >= 1. Výsledkem je přesné **z = y[a] AND y[b]**. Násobení y[a] * y[b] zde nepotřebuješ.

Co znamená >> ?

Jde o indikátorové omezení: **pokud platí levá strana, musí platit pravá**. Vlevo musí být binární proměnná rovná 0 nebo 1. Závorky kolem obou stran piš vždy. Gurobi samo vybere hodnotu choose.

Při choose = 0 vyžadujeme produkci; při choose = 1 spokojenost. Obě podmínky současně jsou také povolené. Jde o „alespoň jedna“, nikoli „právě jedna“.

Přiřazení do výsledného seznamu

```
# Jen po kontrole OPTIMAL; pak zapis cil a tento seznam.
assignment = [-1] * M
for m in range(M):
    for l in range(L):
        if x[m, l].X > 0.5:
            assignment[m] = l
```

10 / Big-M a podmíněná omezení

Zapnutí výroby za jednorázový poplatek

q je nezáporný celočíselný počet výrobků, y říká, zda zaplatíme pronájem. Z kapacity 30 hodin a spotřeby 2 hodin na kus plyne q <= 15. Proto stačí mez 15.

```
q = model.addVar(vtype=gp.GRB.INTEGER, lb=0, name="q")
y = model.addVar(vtype=gp.GRB.BINARY, name="y")
model.addConstr(q <= 15 * y)
model.setObjective(60 * q - 400 * y, gp.GRB.MAXIMIZE)
```

y = 0 vynutí q = 0. Kladné q vynutí y = 1. Samotné omezení ale dovoluje q = 0, y = 1. Pokud chceš přesné „y = 1 právě když vyrábíme“, přidej pro celočíselné q také **q >= y**.

Obecné „pokud y = 1, pak výraz <= b“

```
# Doporuceny jednotny zapis v Gurobi:
model.addConstr((y == 1) >> (expr <= b))

# Alternativni linearni zapis stejne implikace:
model.addConstr(expr <= b + big_m * (1 - y))
```

Vyber jeden zápis. Big-M musí být platná mez: když je podmínka vypnutá, nesmí omezovat jinak přípustná řešení. Pro tento směr stačí big_m >= max(expr - b) nad příslušnými mezemi. Vol co nejmenší platnou hodnotu, ne nekonečno.

NEBO u horníků i bez indikátorů

Protože produkce i spokojenost jsou nezáporné, můžeme u tohoto konkrétního zadání vzít big_m = max(0, U). Pro stejnou proměnnou choose jako na straně 10:

```
big_m = max(0, U)
model.addConstr(production >= U - big_m * choose)
model.addConstr(happiness >= U - big_m * (1 - choose))
```

Při choose = 0 musí produkce dosáhnout U; druhá mez je nejvýše 0. Při choose = 1 se role prohodí. Tuto volbu big_m nepřenášej automaticky na výrazy, které mohou být záporné.

Další použití: pořadí dvou úloh

```
# t jsou nezaporne casy, duration jsou vstupni delky.
# before je binarni: 1 znamena, ze a je pred b.
model.addConstr(
    (before == 1) >> (t[a] + duration[a] <= t[b])
)
model.addConstr(
    (before == 0) >> (t[b] + duration[b] <= t[a])
)
```

11 / Nejčastější chyby a kontrola

Stavba modelu není kontrola hotového řešení

```
# SPATNE pri stavbe modelu:
# if x[i] == 1:
#     model.addConstr(...)

# SPRAVNE: podminka zavisi na vstupnich datech.
if size[i] > capacity[j]:
    model.addConstr(x[i, j] == 0)

# SPRAVNE: podminka zavisi na rozhodovaci promenne.
model.addConstr((y == 1) >> (q <= 10))
```

Další pasti: porovnání je **==**, přiřazení do Pythonové proměnné je **=**. Pro model používej <=, >=, ==. Řetězení **0 <= q <= 10** rozděl na dvě omezení nebo použij lb a ub. Pythonové **and/or** nespojuje Gurobi omezení.

x[i, j] je přístup ke dvouindexové Gurobi kolekci. **x[i][j]** sem nepatří. Známá matice dat jako allowed[i][j] naopak může být běžný seznam seznamů.

Lineární ILP: číslo krát proměnná je v pořádku. Součin dvou rozhodovacích proměnných už není lineární; pro dvě binární použij vzor AND ze strany 10.

Když program běží, ale dává špatný výsledek

1. Vypiš načtené vstupy a zkontroluj indexy 0/1.
2. Ověř význam každé proměnné a směr MINIMIZE/MAXIMIZE.
3. Projdi všechny věty zadání; každá musí mít odpovídající blok.
4. Zkontroluj „nejvýše“ versus „právě“.
5. Přepočítej cíl z nalezeného přiřazení a vstupních dat.
6. Ověř formát výstupu; optimální řešení nemusí být jediné.

```
# Prohlidka skutecne vytvoreneho modelu:
model.write("debug.lp")

# Jen pokud byl model prokazan jako nesplnitelny:
if model.Status == gp.GRB.INFEASIBLE:
    model.computeIIS()
    model.write("conflict.ilp")
```

IIS ukáže konfliktní podmnožinu omezení a mezí. Nemusí být jediná ani mít nejmenší možný počet omezení. Pomáhá najít například chybnou rovnost nebo příliš malou kapacitu.

Prostředí: ModuleNotFoundError pro gurobipy = spouštíš Python bez balíčku. Chyba licence je samostatný problém. Použij školou určený interpreter/prostředí; konkrétní název prostředí se z těchto zadání nedá zjistit. Samotný import nepotvrzuje funkční licenci.

12 / Rychlá karta před testem

```
import gurobipy as gp
model = gp.Model()

x = model.addVars(n, k, vtype=gp.GRB.BINARY, name="x")

for i in range(n):
    model.addConstr(
        gp.quicksum(x[i, j] for j in range(k)) == 1
    )

model.setObjective(
    gp.quicksum(cost[j] * x[i, j]
                for i in range(n) for j in range(k)),
    gp.GRB.MINIMIZE
)
model.optimize()

if model.Status == gp.GRB.OPTIMAL:
    for i in range(n):
        for j in range(k):
            if x[i, j].X > 0.5:
                print(i, j)
```

Toto je modelová kostra přiřazování. Před ní načti n, k, cost; do omezení doplň zbytek zadání a na konci uprav výstup. **Nezaměň tuto kostru za celý hotový program.**

Co si zapamatovat doslova

addVars = více proměnných. **addConstr** = jedno omezení při každém průchodu cyklem. **quicksum** = součet. **setObjective** = cíl. **optimize()** = řeš. **.X** = hodnota proměnné po řešení. **.ObjVal** = hodnota cíle.

Doporučený první trénink

Spusť barvení ze stran 6-7 na třech ukázkách. Potom zkus samostatně jezdce. Nakonec horníky: nejprve přiřazení, pak cechy, přátelství a NEBO. Při každém rozšíření si ručně připrav malý případ, na kterém poznáš, že omezení funguje.

Podklady a ověření syntaxe

Zadání dodaná uživatelem: document.pdf (Knights Placement), ko_test_painting.pdf (Painting the neighborhood) a fotografie Mine of Kuttenberg. Výklad a příklady jsou zpracované pro tento tahák. Referenční dokumentace ověřena 2. 11. 2026:

Gurobi Python Model: metody a indikátory

Gurobi quicksum a globální funkce

Gurobi: omezení a jejich význam

Gurobi: stavy optimalizace